# Logistic Regression: The Sigmoid Function

## Contents


1. Overview . . . 2
2. The new idea: every point pushes or pulls . . . 2
   - 2.1 Push and pull . . . 2
   - 2.2 How strongly . . . 3
3. Why the step function cannot do this . . . 3
4. The sigmoid function . . . 4
5. Sigmoid as a probability . . . 5
   - 5.1 The new prediction . . . 6
   - 5.2 One feature: an S-curve through the data . . . 6
   - 5.3 A map of probabilities . . . 7
6. The new update in action . . . 9
   - 6.1 Four points . . . 9
   - 6.2 Strength and distance . . . 10
7. Does it help? . . . 12
8. Summary . . . 16
9. Sources . . . 16
10. Key terms . . . 16

**Where this fits:** Pipeline map step 8 (Model). See the Course map.

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 Foundations | 1 Frame | 2 Get data | 3 Understand | 4 Clean | 5 Features | 6 Reduce |
| 7 Split | **8 Model** | 9 Evaluate | 10 Tune | 11 Deploy | 12 Test | 13 Monitor |

- **Builds on:** Classification problems; Model-based learning; Gradient descent; Polynomial features; Convex sets and convex optimisation; Log loss (binary cross entropy); Perceptron trick.
- **Leads to:** Softmax regression; Gradient boosting; Perceptron; Multi-layer perceptron (MLP); Backpropagation; Vanishing gradient; Activation functions; Binary Bayes filter.
- **Compare with:** Support vector machines; Tanh.

## 1. Overview

**Key point:** The **perceptron trick** (G-1485) stops as soon as every point is correct. The fix: let every point act on the line, with strength depending on its distance. Replacing the step function with the sigmoid function does exactly this, and also turns the model's output into a probability.

The perceptron trick has a weakness, found when the decision boundary stops too early: once no point is misclassified, the line stops moving, wherever it happens to be. **Logistic regression** (G-1120) keeps improving the line after that.

This Note changes the perceptron's rule so that correctly classified points also take part. The change needs one new ingredient, the **sigmoid function** (G-1798), one of the most important functions in machine learning and deep learning.

## 2. The new idea: every point pushes or pulls

**Key point:** Misclassified points pull the line towards themselves; correctly classified points push it away. The strength depends on how far the point is from the line.

### 2.1 Push and pull

**Key point:** Old rule: only misclassified points act. New rule: all points act.

This is the **push and pull** (G-1592) idea. So far, only misclassified points acted: each one **pulled** the line towards itself. Correct points did nothing.

The new rule: correctly classified points **push** the line away from themselves. Then, even when every point is correct, the line keeps moving: points on both sides push it, and it keeps moving until the pushes balance. The aim is a line in the middle of the gap; Section 7 measures how close this rule gets.

### 2.2 How strongly

> **Key point:** Near correct points push hard, far ones gently. Near misclassified points pull gently, far ones hard.

The strength of each push or pull depends on the point's distance from the line:

| Point | Near the line | Far from the line |
|---|---|---|
| Correctly classified | strong push | weak push |
| Misclassified | weak pull | strong pull |

The table matches common sense. A correct point right next to the line is at risk, so it should push hard. A correct point far away is safe, so it hardly needs to act. A badly misclassified point, far on the wrong side, is a big error, so it should pull hard.

## 3. Why the step function cannot do this

> **Key point:** With the step function, $\hat{y}$ is 0 or 1, so for every correct point $y - \hat{y} = 0$ and the update vanishes.

The update rule from the **perceptron trick** (G-1485) is

$$w_{\text{new}} = w_{\text{old}} + \eta\,(y - \hat{y})\,x$$

For a correctly classified point, $y$ and $\hat{y}$ are equal (both 0 or both 1), so $y - \hat{y} = 0$ and nothing changes. To let correct points act, $y - \hat{y}$ must not be 0.

$y$ is the true class from the data, so it cannot change. What can change is how the model produces $\hat{y}$. So far it computed $z = w \cdot x$, the weights multiplied by the features and added up. For weights $w = (-5, 0.5, 0.01)$ and a student $x = (1, 7.5, 110)$ (the leading 1 pairs with $w_0$):

$$z = (-5)(1) + (0.5)(7.5) + (0.01)(110)$$

$$z = -5 + 3.75 + 1.1 = -0.15$$

The model then passed it through a **step function** (G-1889): 1 if $z > 0$, otherwise 0. As long as $\hat{y}$ is only ever 0 or 1, the problem remains. We need a function whose output can be anything between 0 and 1.

Figure 1 shows the problem as a picture. With the step function, the update $y - \hat{y}$ is exactly 0 for every correctly placed point, near or far, and exactly $\pm 1$ for every misplaced one, however far it is from the line. The planned table of Section 2.2 needs a smooth curve instead.

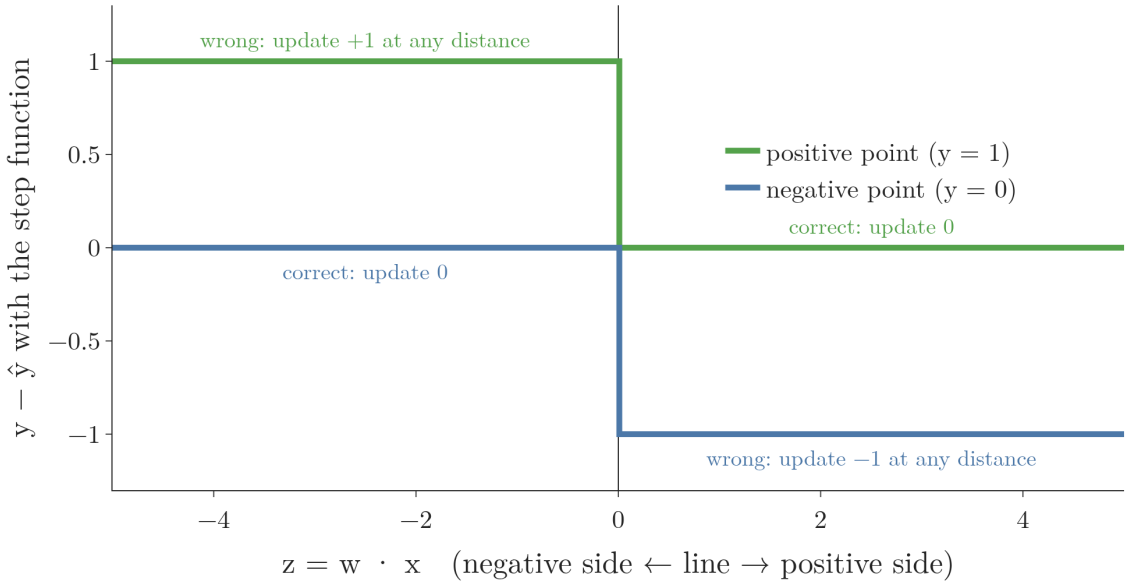



**Figure 1:** The update y − $\hat{y}$ with the step function, against the point's position z = w · x. Correct points give 0 at any distance; wrong points give +1 (positive points, green) or −1 (negative points, blue) at any distance.

## 4. The sigmoid function

> **Key point:** $\sigma$(z) = 1 / (1 + e^(−z)). The sigmoid squeezes any number into the range 0 to 1, with $\sigma(0) = 0.5$.

The **sigmoid function**, also called the **logistic function** (G-1119), is

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

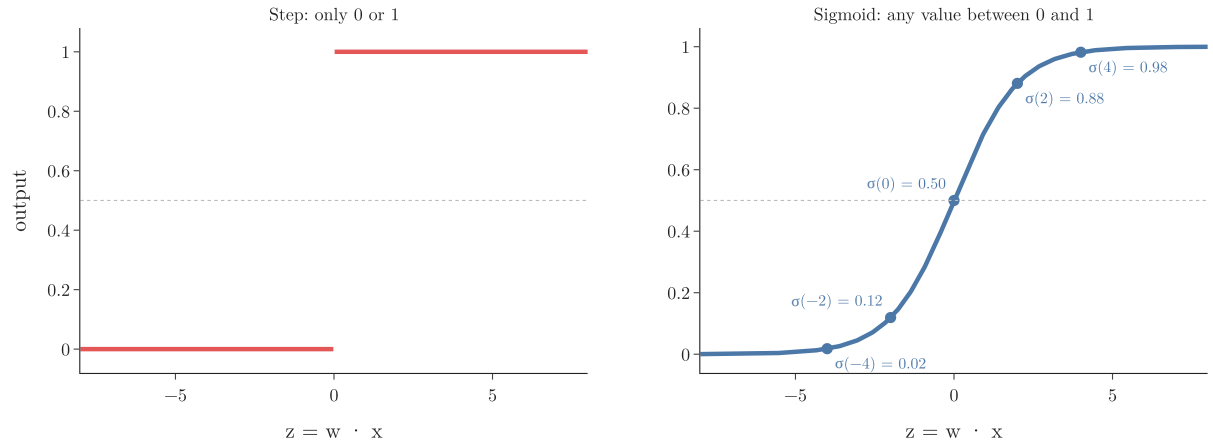



**Figure 2:** The step function and the sigmoid function

Its key properties (Figure 2, right):

- For a very large $z$, $e^{-z}$ is almost 0, so $\sigma(z)$ is almost 1.
- For a very negative $z$, $e^{-z}$ is huge, so $\sigma(z)$ is almost 0.
- At $z = 0$, $e^0 = 1$, so

$$\sigma(0) = \frac{1}{1+1} = 0.5$$

| $z$ | $-4$ | $-2$ | $0$ | $2$ | $4$ |
|---|---|---|---|---|---|
| $\sigma(z)$ | 0.018 | 0.119 | 0.5 | 0.881 | 0.982 |

However large or small the input, the output always stays strictly between 0 and 1. The sigmoid never actually reaches 0 or 1. The S shape gives the function its name: "sigmoid" means S-shaped (Bishop §4.2).

## 5. Sigmoid as a probability

> **Key point:** $\sigma(\mathrm{w} \cdot \mathrm{x})$ can be read as the probability that the point belongs to the positive class. The probability is 0.5 on the line and rises towards 1 further onto the positive side.

### 5.1 The new prediction

> **Key point:** Compute z = w · x as before, but pass it to the sigmoid instead of the step.

For a student with CGPA 7.5 and IQ 110, the model still computes the weighted sum

$$z = w_0 + w_1 \times 7.5 + w_2 \times 110$$

But now $\hat{y} = \sigma(z)$:

- $z > 0$ (positive side): $\hat{y} > 0.5$;
- $z < 0$ (negative side): $\hat{y} < 0.5$;
- $z = 0$ (on the line): $\hat{y} = 0.5$.

To make a yes-or-no prediction, we predict "placed" when $\hat{y} \geq 0.5$, which is the same decision as before.

### 5.2 One feature: an S-curve through the data

> **Key point:** With one feature, the model is the S-curve itself. $w_0$ shifts the curve left or right and $w_1$ sets how steep it is.

Start with one feature only. Figure 3 plots 100 students by CGPA: each dot sits at 1 if the student was placed and at 0 if not. Low CGPAs are almost all at 0, high CGPAs almost all at 1, and the two groups overlap around CGPA 6.

The model is the S-curve

$$\hat{y} = \sigma(w_0 + w_1 \times \text{CGPA})$$

drawn through these dots. Watch the two weights do different jobs:

- $w_0$ **shifts the curve.** The curve crosses 0.5 where the weighted sum is 0, that is at

  $$\text{CGPA} = -\frac{w_0}{w_1}$$

  With $w_1 = 1$, moving $w_0$ from $-4.5$ to $-6.01$ moves the crossing from CGPA 4.5 to 6.01.

- $w_1$ **sets the steepness.** A larger $w_1$ makes the probability climb from near 0 to near 1 over a narrower range of CGPA.

The last frame is the curve that **logistic regression** (G-1120) fits to this data: $w_0 = -39.26$, $w_1 = 6.53$. To predict, read the curve: a student with CGPA 6.5 has

$$z = -39.26 + 6.53 \times 6.5 = 3.19$$

$$\sigma(3.19) = 0.96$$

a 96% chance of being placed. The crossing at CGPA 6.01 is the **decision boundary** (G-555): above it the model predicts placed, below it not placed.

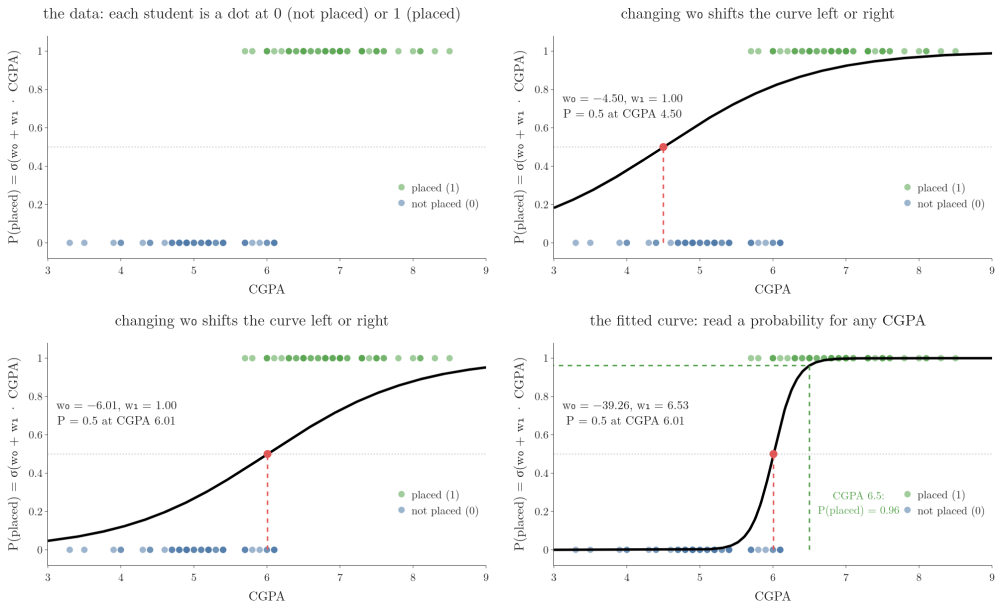



**Figure 3:** The sigmoid on one feature: 100 students at 0 (not placed) or 1 (placed) against CGPA. First $w_0$ shifts the curve, then $w_1$ steepens it; the last frame is the fitted logistic regression curve, which crosses 0.5 at CGPA 6.01. Idea after StatQuest, "Logistic Regression".

### 5.3 A map of probabilities

> **Key point:** Lines parallel to the **decision boundary** (G-555) have equal probability; the probability changes gradually across the plane.

With two features the crossing point becomes the line on which $w \cdot x$ is 0, and the S-curve becomes a surface that rises across it. Figure 4 draws this surface for the line $x_1 + x_2 = 0$. The function is $P(x_1, x_2) = \sigma(x_1 + x_2)$, so $P(0, 0) = 0.5$ and $P(2, 1) = \sigma(3) = 0.95$. Its surface is a ramp with an S-shaped cross-section: flat and low (near 0) on one side of the line, flat and high (near 1) on the other, and steepest on the line, where the height is exactly 0.5 (left panel). Thin lines on the surface join points of the same height.

The right panel is the same surface seen from above, a contour map: the plane coloured by $\sigma(x_1 + x_2)$, with each line joining points of equal probability. Lines close together mean a steep part of the ramp, and the thick black line, where $P = 0.5$, is the line $x_1 + x_2 = 0$ itself.

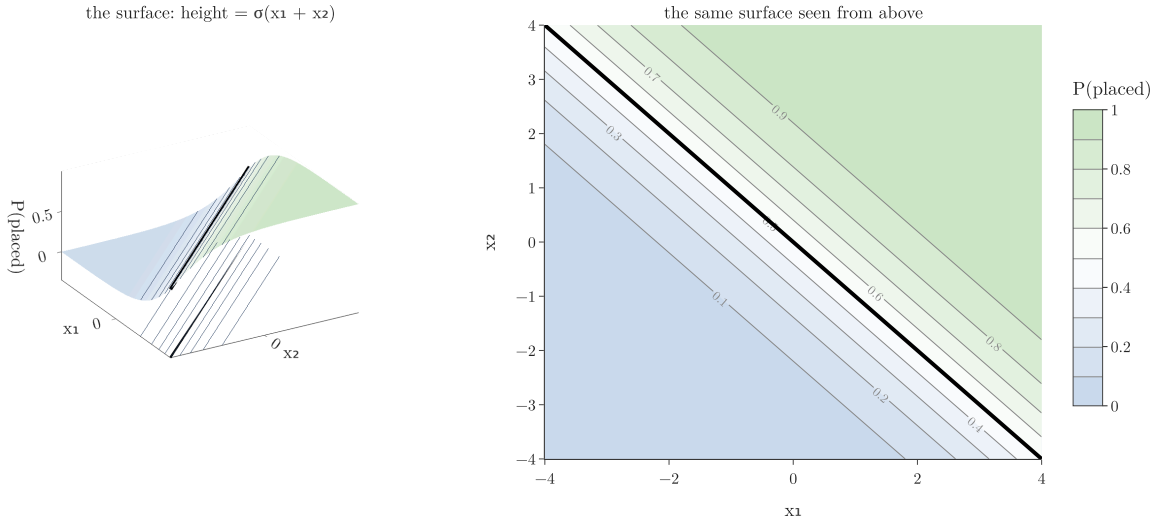



**Figure 4:** $\sigma(\mathrm{w} \cdot \mathrm{x})$ as the probability of being placed: the surface (left) and the same surface seen from above (right). The black line is $\sigma = 0.5$

- On the line itself, $\sigma = 0.5$: a 50% chance of being placed.
- Moving onto the positive side, the value rises through 0.6, 0.7, 0.8 and 0.9, along lines parallel to the boundary. Far away it approaches 1.
- On the negative side it falls towards 0.

So instead of a hard yes or no, every student now gets a **probability of being placed**: the further onto the positive side, the higher the chance. The probability of **not** being placed is $1 - \hat{y}$. A student with $\hat{y} = 0.7$ has a 70% chance of being placed and a 30% chance of not.

This **probabilistic interpretation** (G-1566) is the second view of **logistic regression** (G-1120) mentioned in the **perceptron trick** (G-1485), and log loss builds on it.

**Extra:** What does the number $z = w \cdot x$ itself mean? The **odds** (G-1376) of an event are its probability divided by the probability of the opposite: $p/(1-p)$. A 0.75 chance of being placed is odds of

$$\frac{0.75}{0.25} = 3$$

"3 to 1". The **log-odds** (G-1116) are the natural log of the odds. Solving $p = \sigma(z)$ for $z$ gives

$$z = \ln \frac{p}{1-p}$$

so $z$ is exactly the log-odds of the positive class.

| Probability $p$ | Odds $p/(1-p)$ | Log-odds $z$ |
|---|---|---|
| 0.5 | 1 | 0 |
| 0.731 | 2.72 | 1 |

| | | |
|---|---|---|
| 0.881 | 7.39 | 2 |
| 0.953 | 20.1 | 3 |

Figure 5 draws the fitted CGPA model both ways. As a probability it is an S-curve. As log-odds it is the straight line $w_0 + w_1 \times \text{CGPA}$. So logistic regression is a linear model for the log-odds: $w_1 = 6.53$ means each extra CGPA point adds 6.53 to the log-odds of being placed, and $w_0$ is the log-odds at CGPA 0. The decision boundary is where the log-odds are 0, that is where the odds are 1 to 1 (StatQuest, "Logistic Regression Details Pt1: Coefficients"). Log-odds return in gradient boosting for classification.

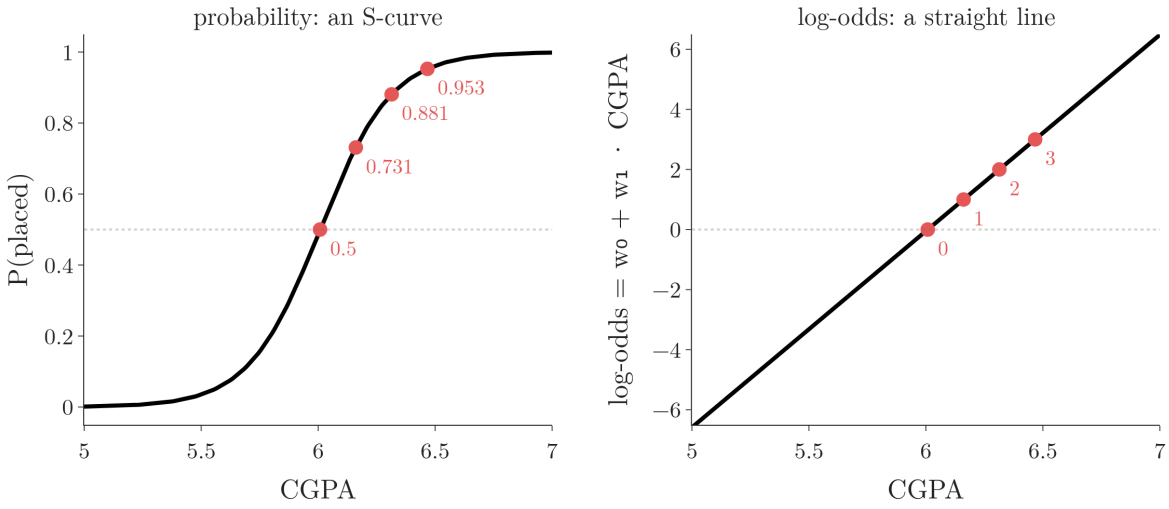



**Figure 5:** The fitted CGPA model drawn two ways. Left: the probability of being placed, an S-curve. Right: the log-odds, a straight line. The red points mark the same four CGPA values in both panels: probabilities 0.5, 0.731, 0.881 and 0.953 are log-odds 0, 1, 2 and 3.

## 6. The new update in action

**Key point:** With $\hat{y} = \sigma(z)$, $y - \hat{y}$ is never exactly 0, so every point moves the line, by an amount that depends on its distance.

### 6.1 Four points

**Key point:** Correct points give small values of $y - \hat{y}$ that push; misclassified points give large values that pull.

Take four points and suppose the model gives these probabilities:

| Point | True $y$ | Side of the line | $\hat{y} = \sigma(z)$ | $y - \hat{y}$ | Effect |
|---|---|---|---|---|---|
| A | 1 | positive (correct) | 0.80 | $+0.20$ | small push |
| B | 0 | negative (correct) | 0.30 | $-0.30$ | push |
| C | 1 | negative (wrong) | 0.30 | $+0.70$ | strong pull |
| D | 0 | positive (wrong) | 0.65 | $-0.65$ | strong pull |

None of the values is 0, so every point updates the weights.

- **A**: the update is $\eta\,(y - \hat{y})\,x$ with

$$y - \hat{y} = 1 - 0.80 = 0.20$$

  so $w$ grows by $0.20\,\eta\,x$. As in the **perceptron trick** (G-1485), adding $x$ moves a positive point further onto the positive side: the line moves away from it, a push.

- **C**:

$$y - \hat{y} = 1 - 0.30 = 0.70$$

  so $w$ grows by $0.70\,\eta\,x$: the same direction but much larger. C sits on the wrong side, so moving it towards the positive side means moving the line towards C, a pull.

- **B** and **D** work the same way with subtraction.

### 6.2 Strength and distance

> **Key point:** y $-$ $\sigma$(z) is large for points deep on the wrong side and small for points deep on the correct side.

Figure 6 draws $y - \sigma(z)$ against $z$, the point's signed position relative to the line.

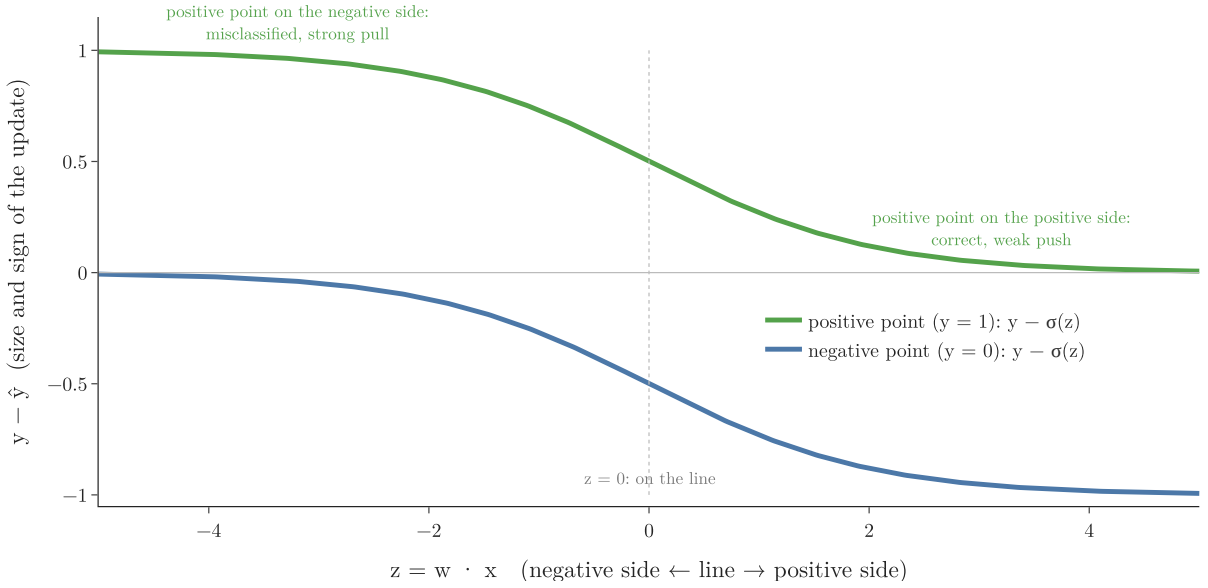



**Figure 6:** The size of the update against the point's position

For a positive point (green, $y = 1$):

- deep on the wrong side ($z = -4$, where $\sigma = 0.018$): $y - \sigma(z)$ is 0.98, a strong pull;
- on the line ($z = 0$, where $\sigma = 0.5$): $y - \sigma(z)$ is 0.5;
- deep on the correct side ($z = 4$, where $\sigma = 0.982$): $y - \sigma(z)$ is 0.018, a very weak push.

For a correct point, the push is strongest when it is just on the correct side, near the line, and fades as it gets further away. The negative points (blue) mirror this. The curve shows exactly the behaviour planned in the table of Section 2.2.

Figure 7 shows the pushes on data. It starts where the step perceptron of the **perceptron trick** (G-1485) stopped on its 24 students: every student is correct, but the line is only 0.14 away from the nearest one. Now the sigmoid rule runs, with learning rate 0.5. Each frame picks one student (red ring) and the arrow shows the push on the line; its length is $|y - \hat{y}|$. Watch the arrow lengths: at pick 1 the student is 0.16 from the line and pushes with strength 0.45; at pick 6 the student is 1.98 away and pushes with only 0.03. Every pick moves the line a little, and after 40 picks the smallest distance to a student has grown from 0.14 to 0.54.

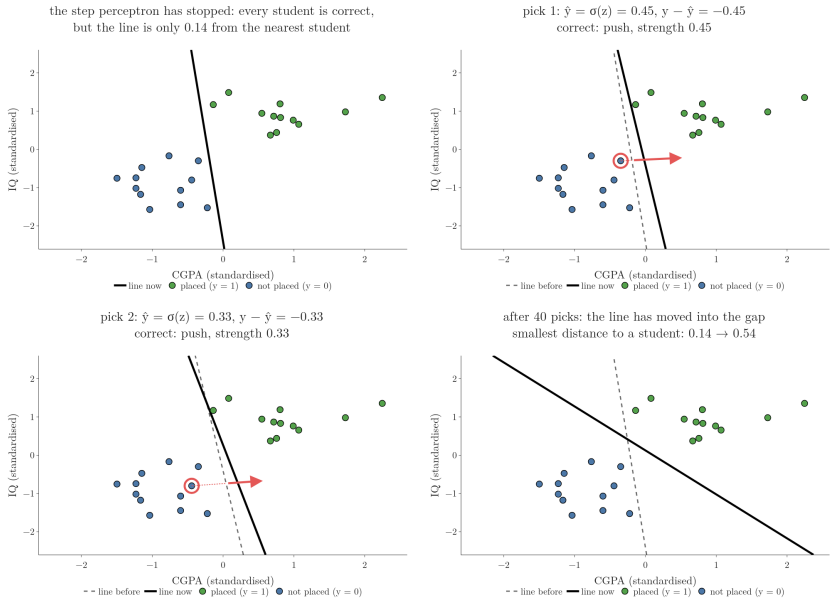



**Figure 7:** The sigmoid rule on 24 students, starting from the line where the step perceptron stopped (dashed in the last frame). Each frame: the picked student (ring) pushes the line away; the arrow's length is the strength $|y - \hat{y}|$. Near students push hard, far students gently. After 40 picks the smallest distance from the line to a student has grown from 0.14 to 0.54.

All 40 picks are pushes, because every student is already on the correct side. The pull on a misclassified student is the perceptron trick's own move, animated in moving the line towards a point.

## 7. Does it help?

**Key point:** The sigmoid version places the line better than the step version, but still not as evenly as **logistic regression** (G-1120).

**Python:** The perceptron with a sigmoid.

```python
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def perceptron(X, y, lr=0.1, loops=1000, seed=0):
    rng = np.random.default_rng(seed)
    X = np.insert(X, 0, 1, axis=1)
    w = np.ones(X.shape[1])
    for _ in range(loops):
        j = rng.integers(0, X.shape[0])
        y_hat = sigmoid(X[j] @ w)              # was: step(X[j] @ w)
        w = w + lr * (y[j] - y_hat) * X[j]
    return w
```

Figure 8 compares three lines on data with a wide gap between the classes (`make_classification` with `class_sep=30`):

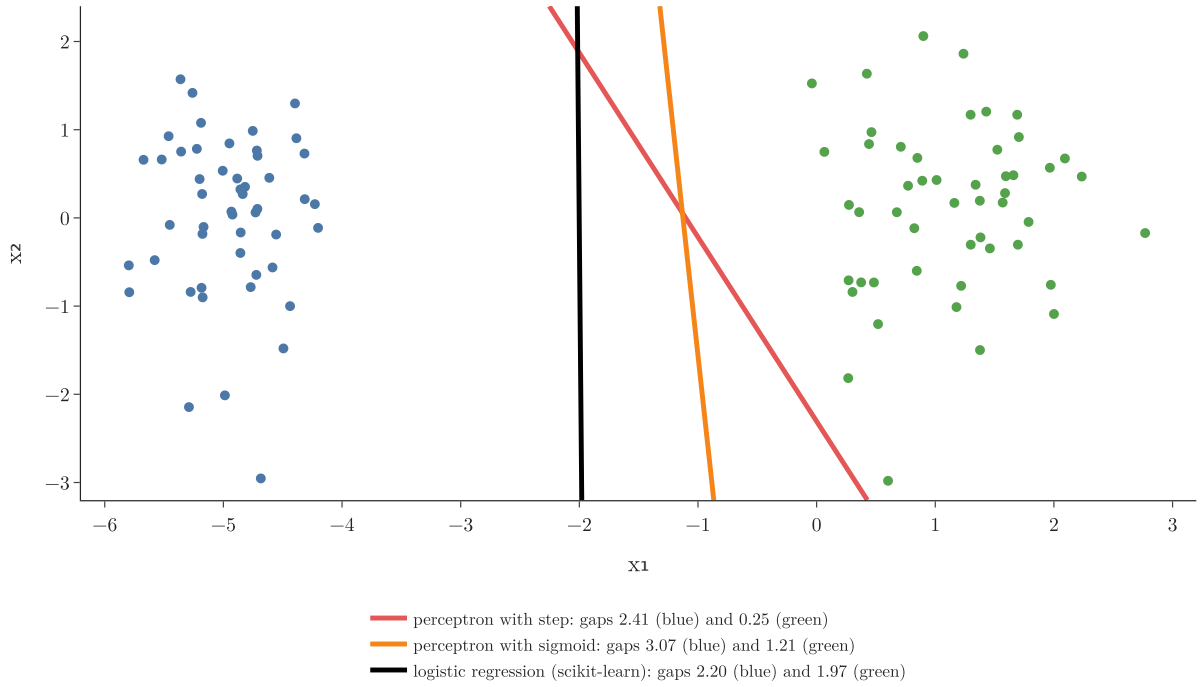



**Figure 8:** Step perceptron, sigmoid perceptron and logistic regression

| Model | Gap to the blue class | Gap to the green class |
|---|---|---|
| Perceptron with step | 2.41 | 0.25 |
| Perceptron with sigmoid | 3.07 | 1.21 |
| Logistic regression (scikit-learn) | 2.20 | 1.97 |

Figure 9 trains both perceptrons side by side on the same random sequence of points. The step perceptron makes its last change at the 44th point; after that every point is correct and the line freezes close to the green class. The sigmoid line keeps moving, because every point still sends an update, and it drifts away from the green points.

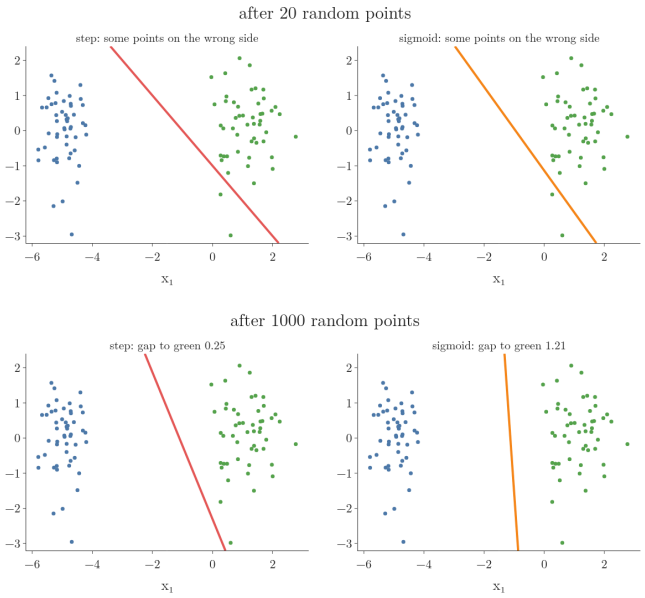



**Figure 9:** The step perceptron (left) and the sigmoid perceptron (right) trained on the same random points, learning rate 0.1. The step line stops changing after 44 points; the sigmoid line keeps moving away from the green class, ending with a gap of 1.21 against 0.25.

The step version stops right next to the green class. The sigmoid version moves well away from it, a clear improvement. But it is still not centred, while scikit-learn's logistic regression keeps a nearly equal gap on both sides.

So the change was in the right direction. Why is the sigmoid line still off-centre? The line is not finished yet. Run longer, the sigmoid perceptron keeps moving towards the logistic regression line, and with the small penalty that scikit-learn adds by default it lands on that line (details in the Extra below). In fact the sigmoid update is already the gradient descent step of logistic regression (Bishop §4.3.2), as the gradient descent will show.

What we still lack is a way to say which line is best: a **loss function** (G-706), one number that measures how good a line is. With a loss function, we know what the updates are minimising and when to stop. The loss function for logistic regression is log loss.

**Extra:** The test, in the notebook. Gaps (blue, green) of the sigmoid perceptron:

| Loops | No penalty | With scikit-learn's default penalty |
|---|---|---|
| 1,000 | 3.07, 1.21 | 2.64, 1.57 |
| 100,000 | 2.61, 1.62 | 2.19, 1.93 |
| 1,000,000 | 2.48, 1.76 | 2.18, 2.00 |

scikit-learn's line has gaps 2.20 and 1.97. Without a penalty, the line keeps drifting slowly: on separable data such as this, the loss that logistic regression minimises (log loss) has no finite best line, and the weights grow for ever (Bishop §4.3.2). The penalty fixes that and gives one definite answer.

## 8. Summary

- Fix for the perceptron: every point acts; correct points push, misclassified points pull, with strength depending on distance, so the line keeps moving after every point is correct and drifts away from the nearest points.
- With the **step function** (G-1889), $y - \hat{y} = 0$ for correct points, so they cannot act, because $\hat{y}$ is only ever 0 or 1.
- The sigmoid maps any $z$ into the range between 0 and 1 (not reaching either), with $\sigma(0) = 0.5$; its formula is in Section 4. So $y - \hat{y}$ is never exactly 0.
- $\sigma(w \cdot x)$ is the probability of the positive class: 0.5 on the line, near 1 deep on the positive side, so every student gets a chance of being placed instead of a hard yes or no.
- Using $\hat{y} = \sigma(z)$ in the update lets every point act and improves the line (gap to the green class 1.21 against 0.25), but not yet to **logistic regression** (G-1120)'s quality, because nothing yet says which line is best; that needs a loss function, log loss.

## 9. Sources

### Built from


- CampusX, "Logistic Regression Part 3 | Sigmoid Function | 100 Days of ML", YouTube, https://www.youtube.com/watch?v=ehO0-6i9qD4
- StatQuest with Josh Starmer, "StatQuest: Logistic Regression", YouTube, https://www.youtube.com/watch?v=yIYKR4sgzI8. The one-feature S-curve through 0/1 data (Figure 3), redrawn on the placement data.
- StatQuest with Josh Starmer, "Logistic Regression Details Pt1: Coefficients", YouTube, https://www.youtube.com/watch?v=vN5cNN2-HWE. The log-odds view of the weights (Section 5.3, Extra).


### Other references


- **Bishop:** Bishop, C. M. *Pattern Recognition and Machine Learning.* Springer, 2006. Section 4.2, p. 197 (the logistic sigmoid; "sigmoid" means S-shaped); Section 4.3.2, pp. 205–207 (logistic regression gradient; separable data).


## 10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

| Term | Meaning |
|---|---|
| Sigmoid function (G-1798) | An S-shaped function that squashes any number into the range 0 to 1, $\sigma(z) = 1/(1 + e^{-z})$; it turns a score into a probability, as in logistic regression. |

| Term | Meaning |
|---|---|
| Push and pull (G-1592) | Moving the line away from a correctly classified point, or towards a misclassified one. |
| Logistic function (G-1119) | Another name for the sigmoid, $1/(1+e^{-z})$: the S-shaped function that squashes any number into a value between 0 and 1, read as a probability. |
| Probabilistic interpretation (G-1566) | Reading the model's output as the probability of the positive class. |
| Perceptron trick (G-1485) | Moving a line towards each misclassified point until the classes are separated. |
| Logistic regression (G-1120) | A classification algorithm that passes a weighted sum of the inputs through the sigmoid to give the probability of a class; the line (or plane) where that probability is 0.5 is the boundary it learns between the classes. |
| Step function (G-1889) | A function that outputs 1 for positive inputs and 0 otherwise; the classic perceptron uses it to turn its score into a class. |
| Decision boundary (G-555) | A line or curve that separates the classes in classification. |
| Odds (G-1376) | How often an event happens divided by how often it does not, e.g. 5 placed to 3 not placed is 5/3; unlike a probability, odds have no upper limit, so evidence updates them by one multiplication, and their log can be predicted with a straight line. |
| Log-odds (G-1116) | The natural log of the odds, $\ln(p/(1-p))$: it turns a probability between 0 and 1 into any number (0 at $p = 0.5$), so a model such as logistic regression can predict it with a straight line, and the sigmoid turns it back into a probability. |
| Loss function (error function) (G-706) | A formula that measures how wrong a model's predictions are; in deep learning, the error on one training row. |